

Iteration 1 Testing Evidence

User journey, automated tests, acceptance criteria and visual evidence

Field	Value
Baseline	Frozen Iteration 1 snapshot, verified 30 August 2026
Evidence refresh	2 September 2026 (Asia/Kuala_Lumpur)
Status	Functional AC: 31 PASS Cross-cutting accessibility: 1 FAIL / open limitation
Scope	E0, E1 and E2 only; active Iteration 2 source excluded

Outcome

The frozen Iteration 1 build is executable and its E0–E2 functional acceptance baseline is supported by a historical authenticated journey, a fresh zero-state/mobile replay, 34 backend tests, 15 iOS tests and configuration validation. All 31 functional acceptance criteria are recorded as PASS. A separate maximum Dynamic Type check is recorded as FAIL because the large editorial headline wraps poorly and the persistent tab bar reduces the visible content area. This limitation is not hidden and was not backported into the frozen baseline.

Area	AC	Result	Evidence
E0 Account & setup	8	PASS	Google OAuth journey, setup, session and account safeguards
E1 Work & care schedule	13	PASS	14-day schedule journey, validation and materialisation tests
E2 Conflict & local alerts	10	PASS	Interval, conflict, handover, failure-state and reminder tests
Mobile / iPhone layout	Cross-cutting	PASS	iPhone 17 Pro, iOS 26.5, zero-state + main + schedule
Accessibility / max Dynamic Type	Cross-cutting	FAIL	Content scrolls, but headline wraps mid-word and tab bar obscures viewport
Invalid and exceptional input	Cross-cutting	PASS	Zero-length, cross-midnight, unknown, missing travel, service failure

Test environment and controls

Control	Value
Source isolation	iteration1-demo-20260830 copied to iteration1-submission; Iteration 2 directories excluded
Mobile target	iPhone 17 Pro simulator profile, iOS 26.5, bundle com.equalpath.ios

Control	Value
iOS toolchain	Full Xcode; Appwrite Apple SDK 18.3.0 from Package.resolved
Backend	Node.js 22-compatible locked packages; Appwrite configuration is read-only during tests
Live-data guardrail	No sign-out, deletion, onboarding replay or mutation of the existing live Appwrite owner dataset
LeanKit guardrail	Read-only verification only; no cards changed

Automated test results

Test ID	Expected	Actual	Result
CFG-01	Appwrite schema validates	15 tables, 168 columns, 37 indexes and 2 Functions validated	PASS
NODE-INITIAL	Backend tests execute	23 passed / 4 failed because two function-local node-appwrite installs were absent	FAIL
NODE-FIX	Repair environment without source change	Ran locked installs in delete-account and iteration1-core packages	PASS
NODE-34	All backend tests pass	34 passed / 0 failed; 171.7 ms on isolated submission copy	PASS
IOS-15	All iOS tests pass	15 passed / 0 failed; Xcode result bundle created	PASS
GH-CI	Both repository CI jobs pass	Backend passed in 14 s; iOS passed in 9 m 15 s; Xcode result uploaded	PASS
SNAP-01	Frozen files retain integrity	Original SHA256SUMS verified after excluding caches and Xcode user state	PASS

Failure and repair record

Environment issue NODE-INITIAL was reproducible: the snapshot intentionally excluded node_modules, while delete-account and iteration1-core each own a package-lock file. Four tests could not import node-appwrite. No product source was changed. Installing the locked dependencies in both function directories resolved the environment defect; the same commands were added to CI and the rerun passed 34/34.

Accessibility issue I1-A11Y-01 remains open in the frozen Iteration 1 submission. At accessibility-extra-extra-extra-large the Tomorrow headline breaks inside words and the tab bar consumes part of the visible viewport. The view is a ScrollView and key controls have accessibility labels, but the visual result does not meet a clean maximum-text-size standard. Fix result: not backported, to preserve the audited Iteration 1 baseline and avoid touching Iteration 2; record as a known limitation and regression target.

Acceptance Criteria matrix

Expected results are taken from the Then clause of the authoritative Given/When/Then source. Actual results are taken from the journey-aligned engineering verification and corroborated by the evidence IDs shown.

E0 — Account and First-Time Setup

AC	Expected result	Actual result	Result	Evidence
0.1.1 Start Google authorisation	EqualPath opens the Appwrite Google OAuth flow in the system/provider experience, displays no password field of its own and does not collect or store the Google password.	The welcome screen has one Google action and calls Appwrite OAuth; EqualPath contains no password form.	PASS	JRN-01; IMG-01; IOS-15
0.1.2 Complete authorisation and reuse one account	the app establishes the Appwrite session, creates at most one owner profile for that Auth user, reuses it on later sign-ins and opens either first-time setup or the existing owner-scoped experience as appropriate.	A successful callback establishes one Appwrite owner profile keyed by Auth user ID; the same session/profile is reused on relaunch or later sign-in.	PASS	JRN-01; IMG-01; IOS-15
0.1.3 Cancel or fail safely	no authenticated data space is opened, no owner profile is created from the failed attempt, a plain-language retry state is shown and Continue with Google remains available.	Cancellation, network, provider and callback failures do not open the authenticated data space and preserve a retry path.	PASS	JRN-01; IMG-01; IOS-15
0.2.1 Capture the owner and children	the app validates required fields, keeps the records owner-scoped and lets her continue without asking for the Google password or editing the authenticated email.	Setup pre-fills the Google identity, keeps the authenticated email read-only and requires at least one named child with an age group.	PASS	JRN-01; IMG-01; IOS-15
0.2.2 Capture weekly routine, travel and support inputs	EqualPath validates each step, describes areas rather than exact live location, performs no live tracking or map lookup and preserves entered values when a step needs correction.	The guided flow captures registered care, work mode and times, broad areas, three manual travel estimates, optional support people and the local-reminder choice without live tracking or a map lookup.	PASS	JRN-01; IMG-01; IOS-15
0.2.3 Save setup and show the first 14-day result	EqualPath writes the owner-scoped profile, children, weekly patterns and support data to Appwrite, materialises dated occurrences across the rolling 14-day horizon, runs the first sweep and shows either the exact first gap result or a truthful no-gap/unknown state; if setup fails, the app retains the setup state and offers a retry instead of claiming completion.	Completing setup writes owner-scoped profile, child, pattern and support data to Appwrite, materialises the 14-day horizon and returns the first verified sweep result; setup failure remains retryable.	PASS	JRN-01; IMG-01; IOS-15
0.3.1 Restore the session and refresh local reminders	the app resumes directly in her owner-scoped experience without repeating Google sign-in or setup and, when local reminders are enabled and iOS permission is granted, rebuilds this iPhone's next 14 days of care-gap reminders for 21:00 on the preceding evening; no APNs token, cloud push or email fallback is used in Iteration 1.	A valid keychain-backed Appwrite session restores directly to the owner experience. Successful sync rebuilds the next 14 days of 21:00 local reminders when permission and the setting are enabled.	PASS	JRN-01; IMG-01; IOS-15

AC	Expected result	Actual result	Result	Evidence
0.3.2 Sign out or delete the account safely	sign-out clears the Appwrite session, local owner state and pending local reminders before returning to the welcome screen; deletion removes the owner's Appwrite data and Auth account only after confirmation, while a failure is reported without falsely claiming the account was deleted.	Sign-out clears session, local owner state and reminders. Confirmed deletion removes owner rows and Auth last; failure cannot appear as successful deletion.	PASS	JRN-01; IMG-01; IOS-15

E1 — Work & Care Schedule Management

AC	Expected result	Actual result	Result	Evidence
1.1.1 Create a work commitment	the app writes the record to her Appwrite account only, refreshes the affected date and displays it with the Work label and source details.	The editor captures date, time, title/type, location, remote flag, and priority, and saves through the authenticated Function.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.1.2 Edit or delete a work commitment	the chosen scope is applied, the rolling plan and affected sweep refresh, and a single-day change is preserved as an override rather than overwritten by later materialisation.	Work entries can be edited/deleted; generated entries expose single-day versus whole-pattern scope and trigger a fresh sweep.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.1.3 Validate work commitment input	the save is blocked with a clear reason and all other entered values remain available for correction.	Validation blocks missing fields, invalid duration, location omissions, and dates outside the rolling 14-day window while retaining draft values.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.2.1 Record a child's required-care window	the app saves the record to Appwrite, materialises it when it is a weekly pattern and displays it on the selected child's schedule.	Owner-scoped children include age group and can receive dated required-care windows.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.2.2 Add childcare coverage	the app places the coverage on the same timeline and merges overlapping coverage before calculating the remaining uncovered portions.	Coverage records provider/carer, child, interval, and notes; overlapping coverage is merged before subtraction.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.2.3 Edit, delete and validate care records	valid changes update the chosen day or pattern scope and trigger an affected-date refresh, while invalid changes are blocked with the rest of the form preserved.	Required-care and coverage entries share validated edit/delete flows and refresh the timeline and conflicts immediately.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.2.4 Record handover responsibilities without inventing certainty	the saved occurrence shows the inbound and outbound handover assignments; missing responsibility or travel inputs remain visibly unassigned or unknown and are never interpreted as proof that the handover is feasible.	Coverage displays drop-off and collection responsibility and uses the three manually entered travel estimates; unassigned or incomplete handover inputs remain visibly unknown rather than no conflict.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.3.1 Show one combined 14-day plan	the app shows all 14 dates, marks dates with detected gaps, uses one consistent Work/Care needed/Coverage legend and lists that date's owner-scoped Appwrite records with weekly-pattern and handover details where applicable.	Schedule provides one 14-day, two-week date surface with a consistent legend and visually marks weekly-pattern occurrences.	PASS	JRN-01; IMG-02; IOS-15; NODE-34

AC	Expected result	Actual result	Result	Evidence
1.3.2 Distinguish covered, uncovered and unknown time	overlapping coverage is counted once, exact remaining uncovered intervals are shown per child, aggregate totals do not hide the per-child source intervals and incomplete evidence remains Unknown rather than No gap.	Exact covered/uncovered intervals are shown after merging; handover conflicts expose endpoint times and responsibility.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.3.3 Keep schedule states current and truthful	the app shows the matching empty, loading or retryable error state and, after a successful refresh, displays the latest verified records without stale or duplicate occurrences.	Loading, empty, load failure, sweep failure/retry, refreshed, and no-schedule-data states are explicit.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.4.1 Create a repeating weekly pattern	the app stores one owner-scoped pattern and creates the corresponding dated occurrences inside the rolling 14-day horizon without requiring a separate save for each day.	The editor creates weekly work, required-care, or coverage patterns with weekdays, time, and effective range.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.4.2 Materialise occurrences idempotently	each pattern-and-date pair produces at most one occurrence, repeated runs create no duplicates and the horizon can advance while preserving existing valid occurrence identity.	Pattern materialisation covers a rolling 14-day window and upserts one `(pattern, date)` occurrence idempotently.	PASS	JRN-01; IMG-02; IOS-15; NODE-34
1.4.3 Edit, skip or delete at the correct scope	a day-only change becomes a preserved override, a skipped day stays skipped, and an edited or deleted pattern affects only its non-overridden occurrences while deliberate exceptions remain intact.	Generated occurrences can be changed/skipped for one day or managed as a whole pattern; pattern updates preserve and report overrides.	PASS	JRN-01; IMG-02; IOS-15; NODE-34

E2 — Conflict Detection, Explanation & Local Alerts

AC	Expected result	Actual result	Result	Evidence
2.1.1 Detect the exact uncovered overlap	the system applies the interval rule $\text{start_A} < \text{end_B}$ AND $\text{start_B} < \text{end_A}$, creates the exact positive-duration overlap for each affected child and retains the work, care and coverage source references used to produce it.	Interval subtraction and strict positive-duration intersection produce exact overlap minutes and source records.	PASS	JRN-01; IMG-03; IOS-15; NODE-34
2.1.2 Handle interval edge cases	touching zero-minute boundaries are not conflicts, overlapping coverage is merged before subtraction, every positive-duration result is split by calendar day and no minute is reported twice.	Tests cover touching, nested, zero-length, partial, and cross-midnight cases; cross-midnight input becomes linked local-day rows.	PASS	JRN-01; IMG-03; IOS-15; NODE-34
2.1.3 Re-detect without duplicates	the system keeps one current record per logical conflict, updates or resolves the previous record rather than appending duplicates and records the latest successful check time.	Conflict IDs use deterministic semantic keys and upsert, so repeated detection retains one current logical result.	PASS	JRN-01; IMG-03; IOS-15; NODE-34

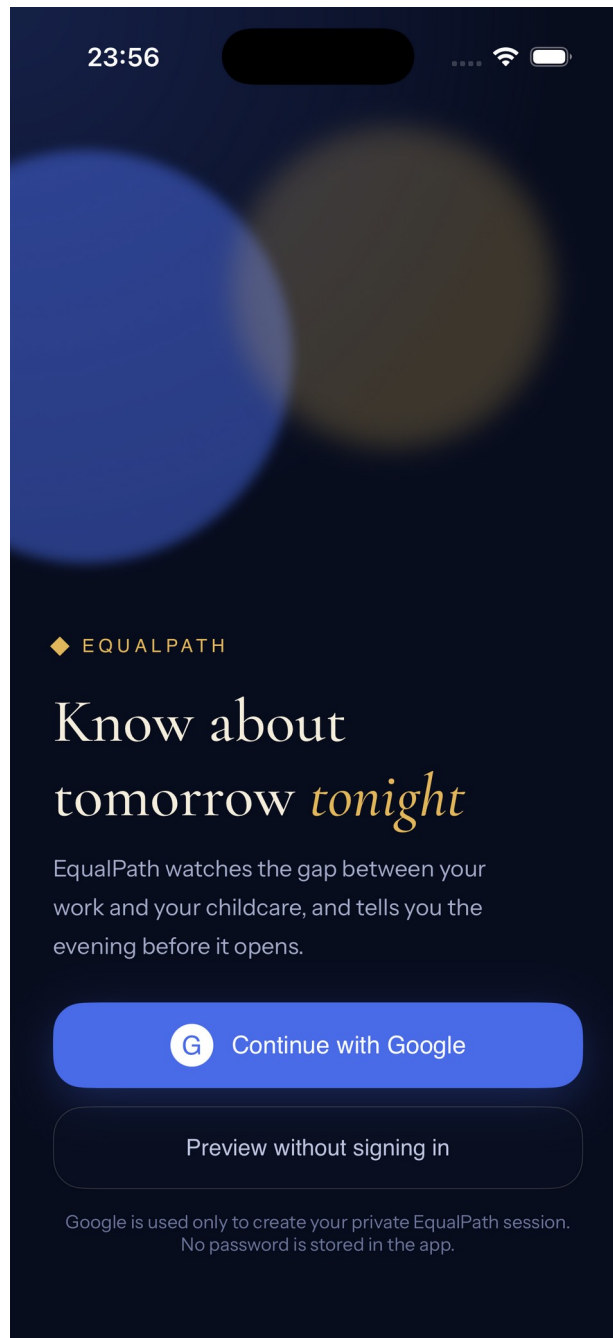
AC	Expected result	Actual result	Result	Evidence
2.1.4 Calculate inbound and outbound handover gaps	a late collection produces the exact interval from the collection boundary to earliest feasible arrival with source handover_out, an impossible morning drop-off produces the symmetric handover_in interval, an exactly feasible boundary produces no handover gap, and missing inputs remain not calculated rather than no conflict.	The engine calculates outbound and inbound handover gaps using responsibility, the relevant work record, and either care ↔ work or home ↔ care travel according to the work location; unknown inputs remain unknown. The third home ↔ work estimate is retained for complete journey context.	PASS	JRN-01; IMG-03; IOS-15; NODE-34
2.2.1 Prioritise conflicts consistently	fixed and non-remote work risk ranks ahead of flexible or remote-capable risk, ties use earliest start and stable record identity, and any aggregate number remains traceable to the per-child gaps it sums.	Fixed, non-remote conflicts rank first, followed by start time and deterministic ID; ordering is tested.	PASS	JRN-01; IMG-03; IOS-15; NODE-34
2.2.2 Explain why a conflict exists	the app shows the child, exact uncovered interval, duration and the Appwrite work, required-care and coverage records that produced it; a handover explanation additionally names its direction and timing inputs, without inventing a cause, vacancy, booking or recommendation.	Conflict explanations name the child, interval, duration, source records, priority rule, and—for handover—the travel input and work boundary used.	PASS	JRN-01; IMG-03; IOS-15; NODE-34
2.2.3 Show truthful states and review-only paths	the app shows the matching uncovered, No gap, Unknown or retry state and never clears a verified warning on failed refresh; any possible path is labelled as a draft for review only and cannot contact, notify or obtain confirmation from another person in Iteration 1.	Stable sorting is deterministic; no-gap is shown only with sufficient evidence. Possible paths are explicitly review-only and cannot contact or confirm another person in Iteration 1.	PASS	JRN-01; IMG-03; IOS-15; NODE-34
2.3.1 Clear a resolved conflict	the affected date is re-swept, the conflict is resolved in the same refresh, the UI shows the verified no-gap result and any obsolete local reminder is removed.	A successful schedule-change scan resolves absent conflicts with `resolution_reason = schedule_change`.	PASS	JRN-01; IMG-03; IOS-15; NODE-34
2.3.2 Update a partially resolved conflict	the same logical conflict is updated to the exact remaining interval and duration, source evidence is refreshed and the local reminder content is rebuilt without creating a duplicate conflict or notification.	Conflict identity excludes mutable interval boundaries, so a partially resolved conflict keeps its identity and updates its remaining interval.	PASS	JRN-01; IMG-03; IOS-15; NODE-34
2.3.3 Preserve the last verified state when refresh fails	the app retains the last verified conflict state, exposes a retryable error, creates no duplicates and never displays or schedules a false cleared/no-gap result; local reminders are changed only from a successful verified sync.	A failed sweep is recorded and cannot clear the last verified conflicts; iOS keeps those results visible with retry.	PASS	JRN-01; IMG-03; IOS-15; NODE-34

Cross-cutting mobile, accessibility and exception tests

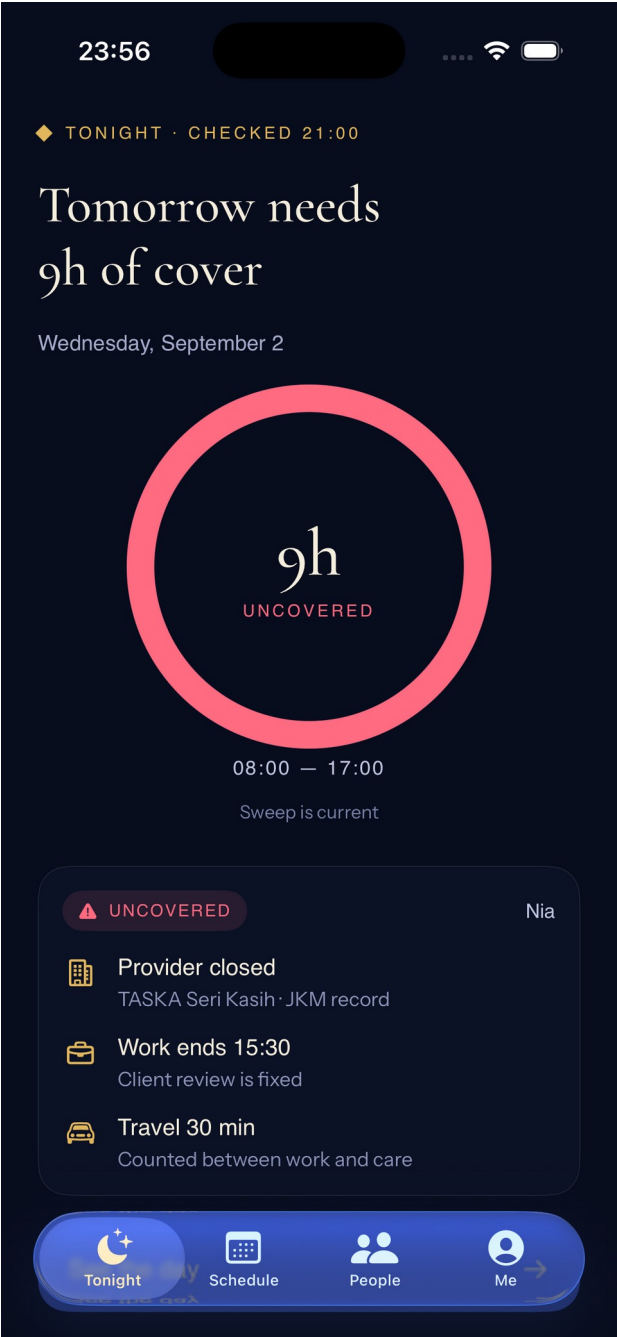
ID	Expected result	Actual result	Result / action
MOB-01	Fresh install renders and permits safe preview on a phone-sized target	Welcome, Tomorrow and Schedule rendered at native 1206 × 2622 output	PASS

ID	Expected result	Actual result	Result / action
A11Y-01	Maximum Dynamic Type remains readable without destructive layout	Scrollable, but headline wraps inside words and the tab bar reduces visible space	FAIL — open
INV-01	Zero-length schedules are rejected	iOS and backend tests reject start == end	PASS
INV-02	Cross-midnight schedules are handled once per local day	Linked local-day spans created; obsolete continuations removed	PASS
INV-03	Missing evidence cannot become No gap	Unknown coverage and missing handover inputs stay Unknown/review	PASS
ERR-01	A failed sweep cannot clear verified conflicts	Backend failure test preserves the prior conflict; UI exposes retry	PASS
SEC-01	Client cannot forge owner identity	Mutation-security tests reject cross-owner discovery/overwrite	PASS

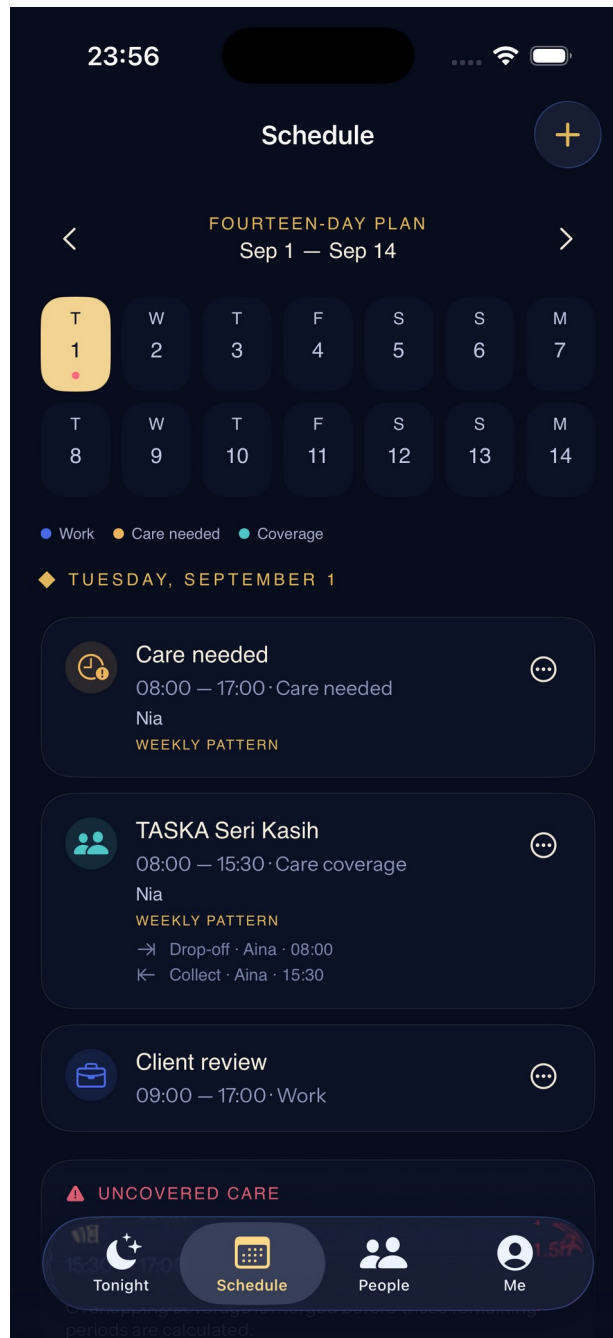
Screenshot evidence



IMG-01 — Fresh-install zero-state welcome screen



IMG-02 — Iteration 1 Tomorrow surface in safe preview mode



IMG-03 — Rolling 14-day Schedule surface



IMG-04 — Maximum Dynamic Type limitation (A11Y-01)

Interpretation limits

- IMG-02 and IMG-03 use deterministic preview data and are visual/mobile evidence, not proof of live Appwrite integration.
- The real Google OAuth and owner-scoped Appwrite journey was completed on 30 August 2026 and is documented in ITERATION1_HANDOVER.md; it was not repeated because the live owner dataset is protected.

- Mentor acceptance remains a human decision. PASS here means the implementation met the documented engineering expectation, not that a mentor has formally accepted the card.

Evidence sources

Public Iteration 1 repository: [Open evidence link](#)

Pinned initial submission commit: [Open evidence link](#)

GitHub Actions CI run: [Open evidence link](#)

Iteration 1 release v0.1.0: [Open evidence link](#)

PGP week4-14 Drive root: [Open evidence link](#)

Existing product screenshots (15 files): [Open evidence link](#)

- Authoritative local source: design-artifacts/EqualPath_Iteration1_Spec.html (3 Epics, 10 User Stories, 31 AC).
- Engineering evidence: appwrite-backend/ITERATION1_VERIFICATION.md and ITERATION1_HANDOVER.md.
- Fresh repeatable evidence: isolated snapshot tests and iPhone 17 Pro / iOS 26.5 simulator screenshots captured on 1–2 September 2026.